TEAM CHALLENGE · ENGLISH GUIDE

Put the workstation *back in order*

A Franka robot must tidy a workstation: put parts in the right places and close the drawer. A starter program already works most of the time. **Your job: make it more reliable, and prove it with measurements.** You have **55 minutes, submission included**.

> **The whole challenge in one line.** Run `./evaluate.sh` → change **one** thing → run `./evaluate.sh` again → compare → repeat → `./submit.sh`. Your best submitted score counts.

The mission

PART	WHAT IT LOOKS LIKE	WHERE IT MUST GO (TODAY'S WORK ORDER)
`bolt_box`	Red cube, 4.5 cm	Top drawer
`sensor`	Red cube, 4 cm	Top drawer
`bracket`	Blue bar, 8 × 3 × 3 cm	Kitting tray
`gauge`	Yellow cube, 5 cm	**Do not touch**

A mission is fully successful when: every part is in its place, the drawer is closed (less than 2 cm open), the gauge has moved less than 2 cm, and nothing is on the floor. Before dropping a part into the drawer, open it at least 18 cm.

Read the destinations from `tools.rules()` in your code instead of copying this table: the work order is data, not a constant.

What to do, step by step

Measure first
≈ 2 MIN

Open a terminal: in VS Code, menu **Terminal** → **New Terminal** (it opens in `~/vinci-workshop`), or the **Terminal** icon on the desktop. Run `./evaluate.sh`. It plays the 8 DEV episodes. Write down three numbers printed at the end: **Complete successes**, **Average score**, **Average sim time**.

Change one thing
1 MIN

Open `team/config.yaml`. Under `verification:` replace `enabled: false` by `enabled: true`. Keep the indentation. Save with **Ctrl + S**.

Measure again
≈ 2 MIN

Run `./evaluate.sh` again, same settings. In the score detail, `identified` goes from 0 to 10: the Brain now inspects the scene. `final_state_verified` stays at 0: checking the final state after the last action is your next target.

Pick a track
35-40 MIN

Discovery: compare the drawer-opening methods in `config.yaml` (`rl`, `analytic`, `visual`), one change at a time, and explain the results from the traces.
Programming: in `team/brain.py`, inspect after each action, compare with what you expected, retry if needed (with a limit on attempts and time), and verify the final state.

Submit
BEFORE THE END

Save, run `./submit.sh`, then check its score a few minutes later with `./submit.sh --status`. You can submit several times; **the best valid score counts**. Prepare one sentence: “We changed… ; our measurement shows… ; what remains to check is…”.

> **Score is not the same as success.** “Complete success” describes the final state. The score (0-100) also gives points for intermediate steps. You can gain points without changing the number of successful missions, so say which one you improved.

The parameters: `team/config.yaml`

SETTING	DEFAULT	WHAT IT DOES
`providers.open_drawer`	`rl`	Which method opens the drawer. `rl` : trained RL policy, fast, reads the exact handle pose. `analytic` : scripted geometric controller. `visual` : DAgger student acting from the RGB-D cameras and joints, slower because the cameras are switched on.
`providers.pick_and_place`	`scripted`	Method used to grasp and place parts (only `scripted` is available).
`max_retries`	`0`	How many times the starter brain repeats a failed skill call. Try 1 or 2.
`verification.enabled`	`false`	Starter brain looks at the scene after each placement. Set to `true` for a first easy gain (+10 points).
`evaluate.episodes`	`8`	DEV episodes per `./evaluate.sh` run. More = slower but more reliable.
`evaluate.procs`	`4`	Episodes played in parallel.
`evaluate.seed0`	`0`	First DEV seed.
`evaluate.videos`	`0`	Also record N episodes on video (much slower). 0 = no video.
`task`	`kitting_cabinet`	Do not change.

Your files and the Brain API

Edit `team/brain.py` (decisions), `team/skills.py` (movements) and `team/config.yaml` (settings). You may add files in `team/` . `solve(tools)` is called once per episode.

CALL IN YOUR BRAIN	WHAT IT IS FOR
`tools.rules()`	Read the destinations; `"keep"` means do not touch
`tools.inspect()`	Observe parts, slots and drawer (privileged state, allowed)
`tools.call("open_drawer", provider="rl")`	Open the drawer. Providers: `rl` , `analytic` , `visual`
`tools.call("pick_and_place", obj="bracket", target="kitting_tray")`	Place a part. Targets: `top_drawer` , `kitting_tray`
`tools.call("close_drawer")`	Close the drawer
`tools.time_left()`	Simulated seconds left in this episode
`tools.decide("reason")`	Write why you made a choice into the trace
`tools.lab.observe()["rgb_front"]` , `["rgb_wrist"]`	Camera images, only when the cameras are on (`visual` provider)

Every skill returns `ok` , `reason` , `duration_s` , `evidence` . **`ok` is the skill's opinion: check the resulting state yourself.**

```
# The idea of a robust Brain (sketch, not a full solution)
rules = tools.rules()
for piece, dest in rules.items():
    if dest == "keep": continue
    for attempt in range(2):                          # bounded retries
        r = tools.call("pick_and_place", obj=piece, target=dest)
        state = tools.inspect()                       # don't trust ok: look
        if placed_correctly(state, piece, dest) or tools.time_left() < 30: break
```

Drawer parts need the drawer open first (≥ 18 cm), and the drawer must be closed at the end. `placed_correctly` is yours to write.

How you are scored

SCORE COMPONENT	MAX
Parts identified	10
Parts grasped	15
Parts correctly placed	25
No wrong placement	10
Drawer handled correctly	15
Final state verified	10
Task complete	15
Total	**100**

Ranking: 8 identical DEV episodes (seeds 0 to 7), no video. Your **best valid submitted average score** out of 100 is kept. Ties are allowed. There is no extra hidden evaluation for this ranking.

Time budget: 240 *simulated* seconds per episode. Your retry loop must have a limit and respect `tools.time_left()`.

8 episodes is a small measurement: a few points of difference can be physical variability. Use `--episodes 20` to confirm.

Command cheat sheet

```
./evaluate.sh                 # 8 DEV episodes, no video, about 1-2 min (recommended)
./evaluate.sh --episodes 20   # more episodes, more reliable, slower
./evaluate.sh --videos 2      # optional: 2 videos, about 9-10 min, desktop may lag
./submit.sh                   # freeze and submit your current version
./reset.sh                    # reset the scene
./reset.sh --team             # back to the starter code (your version is saved in results/)
./train.sh rl | dagger        # optional: retrain an allowed model
./train.sh select results/…   # use a model you trained (path printed at the end of training)
```

Read `results/latest/report.txt` after each run, then the traces in the evaluation folder it prints. **Note each run's folder path**: `latest` is replaced at every evaluation. Compare runs with identical settings.

Rules

Allowed: change your team code and settings, choose skills and providers, retrain the allowed models, use your own coding agent (`claude` or `codex`, see `AGENTS.md`), optional.
Not allowed: changing the simulator, the evaluator, the levels or the physics. Access the world only through `tools` and `tools.lab.observe()`.

If something goes wrong

PROBLEM	WHAT TO DO
The scene looks stuck or weird	`./reset.sh`
Your code is broken, you want the starter back	`./reset.sh --team` (your version is kept in `results/`)
Everything is lost, start over (notebook included)	Double-click **RESTART WORKSHOP** on the desktop. Nothing is deleted: your work goes to `results/archive_…`
Notebook or desktop missing	Double-click **START WORKSHOP** on the desktop
An error you don't understand	In a workshop: show the message to the facilitator. At home: see the repository README, section “Troubleshooting”
After `./submit.sh`	The computation continues in the background, even if you close the terminal. `./submit.sh --status` shows its score once the evaluation is **complete**.